

Gestionale Cinema

Contesto

Una catena di cinema vuole gestire internamente la programmazione dei film nelle varie sale.

Il sistema deve distinguere diversi tipi di proiezione:

1. **Proiezione standard**
 - normale proiezione di un film.
2. **Proiezione 3D**
 - proiezione con supplemento per occhiali 3D.
3. **Evento speciale**
 - anteprima, rassegna, incontro con il regista, film restaurato, evento limitato.

Tutte le proiezioni hanno informazioni comuni, ma alcune hanno informazioni specifiche.

1. Interfaccia **Identificabile**

Crea un'interfaccia per tutti gli oggetti che hanno un id.

Metodi:

- `int getId();`

Dovranno implementarla:

- **Film**
- **Sala**
- **Proiezione**
- eventualmente **Prenotazione**, se vuoi aggiungerla.

2. Interfaccia **Programmabile**

Crea un'interfaccia per tutti gli oggetti che hanno una data e un orario di programmazione.

- LocalDate getData();
- LocalTime getOraInizio();
- LocalDateTime getDataOraInizio();

Questa interfaccia sarà implementata dalla classe astratta **Proiezione**.

3. Interfaccia **Prezzabile**

Crea un'interfaccia per gli oggetti che possono calcolare un prezzo.

- double calcolaPrezzoFinale();

La implementerai nelle proiezioni.

4. Interfaccia funzionale generica **Filtro<T>**

- boolean accetta(T elemento);

La userai per filtrare film o proiezioni.

5. Classe **Film**

La classe **Film** rappresenta un film presente nel catalogo del cinema.

Attributi richiesti

```
private int id;  
private String titolo;  
private String regista;  
private int durataMinuti;  
private LocalDate dataUscita;  
private Set<GenereFilm> generi;
```

Il campo **generi** deve essere un **Set<GenereFilm>** perché un film non deve avere generi duplicati.

Dove GenereFilm è un enum con questi possibili valori:

"Azione"

"Drammatico"
"Thriller"
"Fantascienza"
"Animazione"
"Commedia"
"Horror"

Metodi richiesti

```
void aggiungiGenere(GenereFilm genere);  
void rimuoviGenere(GenereFilm genere);  
boolean contieneGenere(String genere);  
boolean isFilmRecente();  
int getAnniDaUscita();  
String getDescrizione();
```

Metodi sulle date

Devi usare `LocalDate`.

`isFilmRecente()`

Restituisce `true` se il film è uscito negli ultimi 2 anni.

`getAnniDaUscita()`

Restituisce quanti anni sono passati dall'uscita del film.

Puoi usare:

```
Period.between(dataUscita, LocalDate.now()).getYears();
```

6. Classe `Sala`

La classe `Sala` rappresenta una sala del cinema.

Attributi richiesti

```
private int id;  
private String nome;  
private int numeroPosti;  
private boolean supporta3D;
```

private Set<String> caratteristiche;

Esempi di caratteristiche:

"Dolby Atmos"

"IMAX"

"Poltrone reclinabili"

"Accessibile disabili"

"Schermo grande"

Metodi richiesti

void aggiungiCaratteristica(String caratteristica);

boolean haCaratteristica(String caratteristica);

String getDescrizione();

Regole:

- **nome** non deve essere vuoto;
- **numeroPosti** deve essere maggiore di 0;
- una caratteristica non deve essere vuota.

7. Classe astratta **Proiezione**

La classe **Proiezione** rappresenta una proiezione generica.

Deve implementare:

Identificabile

Programmabile

Prezzabile

Attributi richiesti

private int id;

private Film film;

private Sala sala;

private LocalDate data;

private LocalTime oraInizio;

private double prezzoBase;

private Set<String> tag;

Esempi di tag:

"serale"
"weekend"
"famiglie"
"anteprima"
"3d"
"evento"
"lingua originale"

Metodi richiesti

```
void aggiungiTag(String tag);  
boolean contieneTag(String tag);  
LocalDateTime getDataOraInizio();  
LocalDateTime getDataOraFine();  
boolean isOggi();  
boolean isNelWeekend();  
boolean isSerale();  
boolean isTerminata();  
String getDettagliBase();  
abstract String getTipoProiezione();
```

Metodi sulle date

getDataOraInizio()

Deve combinare `LocalDate` e `LocalTime`.

```
LocalDateTime.of(data, oraInizio);
```

getDataOraFine()

Deve calcolare quando finisce la proiezione usando la durata del film.

Esempio:

```
getDataOraInizio().plusMinutes(film.getDurataMinuti());
```

- isOggi()

Restituisce `true` se la proiezione è programmata per oggi.

- isNelWeekend()

Restituisce `true` se la proiezione è sabato o domenica.

Puoi usare:

`data.getDayOfWeek()`

- `isSerale()`

Restituisce `true` se la proiezione inizia dalle `20:00` in poi.

- `isTerminata()`

Restituisce `true` se la data e ora di fine sono precedenti a `LocalDateTime.now()`.

8. Classe `ProiezioneStandard`

Estende `Proiezione`.

Non ha molti attributi aggiuntivi.

Metodo richiesto

```
public double calcolaPrezzoFinale()
```

Logica:

- prezzo finale = prezzo base;
- se è weekend, aggiungi `2.00`;
- se è serale, aggiungi `1.50`.

9. Classe `Proiezione3D`

Estende `Proiezione`.

Attributi specifici

```
private double supplemento3D;  
private boolean occhialiInclusi;
```

Metodo richiesto

```
public double calcolaPrezzoFinale()
```

Logica:

- prezzo finale = prezzo base;
- aggiungi supplemento 3D;
- se è weekend, aggiungi 2.00;
- se gli occhiali non sono inclusi, aggiungi 1.00.

Regola

Se la sala non supporta il 3D, stampa un messaggio di errore.

10. Classe **EventoSpeciale**

Estende **Proiezione**.

Attributi specifici

private String nomeEvento;

private String ospite;

private boolean postiLimitati;

Metodo richiesto

public double calcolaPrezzoFinale()

Logica:

- prezzo finale = prezzo base;
- aggiungi 5.00 perché è un evento speciale;
- se ci sono posti limitati, aggiungi 3.00;
- se è serale, aggiungi 1.50.

11. Classe generica **Archivio<T extends Identificabile>**

Crea una classe generica per archiviare oggetti identificabili.

- public class Archivio<T extends Identificabile>

Internamente deve usare una `Map<Integer, T>`.

- private Map<Integer, T> elementi;

Metodi richiesti

void aggiungi(T elemento);

T cercaPerId(int id);

boolean rimuoviPerId(int id);

List<T> trovaTutti();

int contaElementi();

Questa classe deve funzionare per:

Archivio<Film> archivioFilm = new Archivio<>();

Archivio<Sala> archivioSale = new Archivio<>();

Archivio<Proiezione> archivioProiezioni = new Archivio<>();

12. Classe **GestoreCinema**

Attributi richiesti

private Archivio<Film> archivioFilm;

private Archivio<Sala> archivioSale;

private Archivio<Proiezione> archivioProiezioni;

private List<Proiezione> programmazione;

private Map<LocalDate, List<Proiezione>> proiezioniPerData;

private Map<Integer, List<Proiezione>> proiezioniPerSala;

private Map<String, List<Film>> filmPerGenere;

13. Metodi richiesti in **GestoreCinema**

Aggiungere film

`void aggiungiFilm(Film film);`

Il metodo deve:

- aggiungere il film all'archivio;
- aggiornare la `Map<String, List<Film>> filmPerGenere`.

Esempio:

"Thriller" -> lista di film thriller

"Commedia" -> lista di film commedia

Aggiungere sala

`void aggiungiSala(Sala sala);`

Il metodo deve aggiungere la sala all'archivio delle sale.

Aggiungere proiezione

`void aggiungiProiezione(Proiezione proiezione);`

Il metodo deve:

- aggiungere la proiezione all'archivio;
 - aggiungerla alla `List<Proiezione> programmazione`;
 - aggiornare `Map<LocalDate, List<Proiezione>>`;
 - aggiornare `Map<Integer, List<Proiezione>>`.
-

Cercare proiezioni per data

`List<Proiezione> cercaProiezioniPerData(LocalDate data);`

Deve usare:

`Map<LocalDate, List<Proiezione>>`

Cercare proiezioni per sala

List<Proiezione> cercaProiezioniPerSala(int idSala);

Deve usare:

Map<Integer, List<Proiezione>>

Cercare film per genere

List<Film> cercaFilmPerGenere(String genere);

Deve usare:

Map<String, List<Film>>

Cercare proiezioni future

List<Proiezione> cercaProiezioniFuture();

Deve restituire solo le proiezioni che non sono ancora terminate.

Puoi usare:

proiezione.isTerminata()

Cercare proiezioni di oggi

List<Proiezione> cercaProiezioniDiOggi();

Deve restituire solo le proiezioni con data uguale a `LocalDate.now()`.

Cercare proiezioni serali

List<Proiezione> cercaProiezioniSerali();

Deve restituire le proiezioni che iniziano dalle `20:00` in poi.

14. Metodo generico con filtro

Nel `GestoreCinema`, crea:

List<Proiezione> filtraProiezioni(Filtro<Proiezione> filtro);

Il metodo deve scorrere la lista **programmazione** e restituire solo le proiezioni accettate dal filtro.

Esempi di utilizzo nel **main**:

```
List<Proiezione> costose = gestore.filtraProiezioni(p -> p.calcolaPrezzoFinale() > 15);
```

```
List<Proiezione> weekend = gestore.filtraProiezioni(p -> p.isNelWeekend());
```

```
List<Proiezione> eventi = gestore.filtraProiezioni(p -> p instanceof EventoSpeciale);
```

Puoi fare anche un metodo simile per i film:

```
List<Film> filtraFilm(Filtro<Film> filtro);
```

Esempi:

```
List<Film> recenti = gestore.filtraFilm(f -> f.isFilmRecente());
```

```
List<Film> lunghi = gestore.filtraFilm(f -> f.getDurataMinuti() > 140);
```

15. Parte su **instanceof** e casting

Crea nel **GestoreCinema** un metodo:

```
void stampaDettaglioSpecifico(Proiezione proiezione);
```

Il metodo riceve una **Proiezione**, quindi un riferimento generale.

Dentro devi controllare il tipo reale dell'oggetto.

Caso **ProiezioneStandard**

Stampa:

- titolo film;
- sala;
- data;
- ora;
- prezzo finale.

Caso **Proiezione3D**

Usa `instanceof` e casting.

Stampa:

- titolo film;
- sala;
- supplemento 3D;
- se gli occhiali sono inclusi;
- prezzo finale.

Caso **EventoSpeciale**

Stampa:

- nome evento;
- ospite;
- se ha posti limitati;
- prezzo finale.

16. Parte con lambda

Nel `main`, oppure in metodi dedicati, devi fare almeno questi ordinamenti.

Ordinare proiezioni per data e ora

```
programmazione.sort((p1, p2) ->  
p1.getDataOraInizio().compareTo(p2.getDataOraInizio()));
```

Ordinare proiezioni per prezzo finale crescente

```
programmazione.sort((p1, p2) -> Double.compare(p1.calcolaPrezzoFinale(),  
p2.calcolaPrezzoFinale()));
```

Ordinare film per durata decrescente

```
film.sort((f1, f2) -> Integer.compare(f2.getDurataMinuti(), f1.getDurataMinuti()));
```

17. Parte con classe anonima

Crea un'interfaccia Notificatore :

- void inviaNotifica(Proiezione proiezione);

Nel `main`, crea un notificatore con classe anonima.

Il metodo deve stampare una notifica relativa alla proiezione programmata per la sua specifica data e orario

Poi crea anche un notificatore con lambda.

Usali quando aggiungi una nuova proiezione o quando stampi la programmazione.

18. Lettura dei dati da file

In questa versione dell'esercizio, oltre a creare dati manualmente nel `main`, devi aggiungere anche la possibilità di caricare film, sale e proiezioni da file testuali.

Il caricamento da file serve per simulare una situazione più realistica, dove i dati iniziali del gestionale non vengono scritti direttamente nel codice Java, ma importati da file esterni.

Il progetto deve prevedere tre file:

film.txt

sale.txt

proiezioni.txt

I dati devono essere caricati in questo ordine:

1. prima i film;
2. poi le sale;
3. infine le proiezioni.

Questo ordine è importante perché ogni proiezione deve fare riferimento a un film e a una sala già esistenti.

19. Classe **LettoreDatiCinema**

Crea una classe dedicata alla lettura dei file.

Questa classe deve occuparsi di:

- leggere il file dei film;

- leggere il file delle sale;
- leggere il file delle proiezioni;
- creare gli oggetti Java corrispondenti;
- aggiungerli al **GestoreCinema**;
- ignorare le righe non valide;
- stampare un messaggio quando una riga non può essere caricata.

La classe può avere questi metodi:

```
public void caricaFilm(String percorsoFile, GestoreCinema gestore) {

    // legge film.txt }

public void caricaSale(String percorsoFile, GestoreCinema gestore) {

    // legge sale.txt }

public void caricaProiezioni(String percorsoFile, GestoreCinema gestore) {

    // legge proiezioni.txt }
```

20. Formato del file **film.txt**

Il file dei film deve usare questo formato:

id;titolo;regista;durataMinuti;dataUscita;generi

Esempio:

1;Dune Part Two;Denis Villeneuve;166;2024-03-01;FANTASCIENZA,AZIONE

2;Oppenheimer;Christopher Nolan;180;2023-08-23;DRAMMATICO,STORICO

I campi sono separati da ;.

I generi multipli sono separati da ,.

Quindi questa riga:

1;Dune Part Two;Denis Villeneuve;166;2024-03-01;FANTASCIENZA,AZIONE

significa:

- id: 1
- titolo: **Dune Part Two**
- regista: **Denis Villeneuve**

- durata: 166
- data uscita: 2024-03-01
- generi: FANTASCIENZA, AZIONE

21. Formato del file **sale.txt**

Il file delle sale deve usare questo formato:

id;nome;numeroPosti;supporta3D;caratteristiche

Esempio:

1;Sala IMAX;220;true;IMAX,DOLBY_ATMOS,SCHERMO_GRADE

2;Sala Blu;120;false;ACCESSIBILE_DISABILI,POLTRONE_RECLINABILI

Il campo **supporta3D** deve essere scritto come: true o false

Le caratteristiche possono rimanere **String**.

22. Formato del file **proiezioni.txt**

Il file delle proiezioni deve usare questo formato:

id;tipo;filmId;salaId;data;oraInizio;prezzoBase;extra1;extra2;extra3;tag

Il campo **tipo** può assumere questi valori:

STANDARD

TRE_D

EVENTO

Esempi:

1;STANDARD;1;1;2026-06-10;21:30;10.00;-;-;SERALE,WEEKEND

2;TRE_D;1;3;2026-06-11;20:15;12.00;3.00;true;-;TRE_D,SERALE

3;EVENTO;2;1;2026-06-12;19:00;14.00;Anteprima speciale;Regista ospite;true;EVENTO,ANTEPRIMA

23. Significato dei campi del file proiezioni

Campi comuni

Questi campi sono sempre presenti:

id

tipo

filmId

salaId

data

oraInizio

prezzoBase

tag

Esempio:

1;STANDARD;1;1;2026-06-10;21:30;10.00;-;-;SERALE,WEEKEND

significa:

- id proiezione: 1
- tipo: STANDARD
- filmId: 1
- salaId: 1
- data: 2026-06-10
- ora inizio: 21:30
- prezzo base: 10.00
- tag: SERALE, WEEKEND

Campi extra per STANDARD

Per una proiezione standard, i campi extra non servono.

Quindi puoi scrivere: -;-;

Esempio:

1;STANDARD;1;1;2026-06-10;21:30;10.00;-;-;SERALE,WEEKEND

Campi extra per TRE_D

Per una proiezione 3D:

extra1 = supplemento3D

extra2 = occhialiInclusi

extra3 = -

Esempio: 2;TRE_D;1;3;2026-06-11;20:15;12.00;3.00;true;-;TRE_D,SERALE

significa:

- supplemento 3D: 3.00
- occhiali inclusi: true

Campi extra per EVENTO

Per un evento speciale:

extra1 = nomeEvento

extra2 = ospite

extra3 = postiLimitati

Esempio: 3;EVENTO;2;1;2026-06-12;19:00;14.00;Anteprima speciale;Regista ospite;true;EVENTO,ANTEPRIMA

significa:

- nome evento: Anteprima speciale
- ospite: Regista ospite
- posti limitati: true

24. Main di test richiesto

Crea una classe Main.

1. Creare almeno altri 2 film

Ogni film deve avere:

- id;
- titolo;
- regista;
- durata in minuti;
- data di uscita;
- almeno 2 generi.

2. Creare almeno altre 3 sale

Ogni sala deve avere:

- id;
- nome;
- numero posti;
- supporto 3D;
- almeno 2 caratteristiche.

3. Creare almeno altre 6 proiezioni

Devi creare:

- almeno 4 `ProiezioneStandard`;
- almeno 3 `Proiezione3D`;
- almeno 3 `EventoSpeciale`.

Ogni proiezione deve avere:

- id;
- film;
- sala;
- data;
- ora inizio;
- prezzo base;
- almeno 1 o 2 tag.

25. Report richiesti

Nel `main`, stampa almeno questi report.

Report 1 — Tutti i film

Per ogni film stampa:

- id;
- titolo;
- regista;
- durata;
- data uscita;
- anni dall'uscita;
- generi.

Report 2 — Tutte le proiezioni

Per ogni proiezione stampa:

- id;
 - titolo film;
 - sala;
 - data;
 - ora inizio;
 - ora fine;
 - tipo proiezione;
 - prezzo finale.
-

Report 3 — Proiezioni di oggi

Usa:

`cercaProiezioniDiOggi()`

Report 4 — Proiezioni future

Usa:

`cercaProiezioniFuture()`

Report 5 — Proiezioni per data

Scegli una data specifica e cerca tutte le proiezioni di quella data.

Esempio:

```
LocalDate dataRicerca = LocalDate.of(2026, 6, 10);
```

Report 6 — Proiezioni per sala

Scegli una sala e stampa tutte le proiezioni associate.

Report 7 — Film per genere

Cerca tutti i film di genere:

"Fantascienza"

oppure:

"Thriller"

Report 8 — Proiezioni serali

Stampa tutte le proiezioni che iniziano dalle 20:00 in poi.

Report 9 — Proiezioni del weekend

Usa il filtro con lambda.

Report 10 — Proiezioni costose

Usa il filtro con lambda.

```
gestore.filtraProiezioni(p -> p.calcolaPrezzoFinale() > 15.0);
```

Report 11 — Dettaglio specifico con **instanceof**

Scorri tutte le proiezioni e chiama:

```
gestore.stampaDettaglioSpecifico(proiezione);
```

Questo serve per verificare il tipo reale dell'oggetto.

Report 12 — Ordinamenti con lambda

Ordina e stampa:

- proiezioni per data e ora;
- proiezioni per prezzo crescente;
- film per durata decrescente.

26. Vincoli di validazione semplici

Non usare eccezioni.

Nei costruttori o nei setter puoi fare controlli semplici.

Per **Film**

- titolo non vuoto;
- regista non vuoto;
- durata maggiore di 0;
- data uscita non nulla;
- genere non vuoto.

Per **Sala**

- nome non vuoto;
- numero posti maggiore di 0;
- caratteristica non vuota.

Per **Proiezione**

- film non nullo;
- sala non nulla;
- data non nulla;

- ora inizio non nulla;
- prezzo base maggiore di 0;
- tag non vuoto.

Per **Proiezione3D**

- se la sala non supporta il 3D, stampa un messaggio di errore;
- puoi comunque creare l'oggetto, ma il messaggio deve segnalare che la scelta non è corretta.